

Installing a Debian 13 Web Server on Qemu/KVM

Apache, PostgreSQL, and PHP — accessible from the host machine

A practical install guide



Khalil Bendref

May 2026

Table of contents

Table of contents	2
About this guide	4
Conventions used in this guide	4
1. Before you start	5
1.1 What you need	5
1.2 A note on the helper scripts	5
2. Install Debian 13	6
2.1 Verify the installer image	6
2.2 Boot the installer	6
2.3 Configure language and location	6
2.4 Set the hostname and user account	7
2.5 Partition the disk	7
2.6 Select the software	8
2.7 Install GRUB and finish	9
2.8 Move the disk image to the shared server	9
3. Verify the base system	10
3.1 Start the VM	10
3.2 Check the filesystem table	10
3.3 Check that SSH is running	11
3.4 Confirm there's no graphical interface	11
3.5 Port forwarding overview	12
4. Install and configure Apache	13
4.1 Install the package	13
4.2 Check the service status	13
4.3 Test the server from inside the VM	13
4.4 Test the server from your Linux host	14
5. Install and configure PostgreSQL	15
5.1 Install the package	15
5.2 Check the service status	15
5.3 Switch to the postgres user	15
5.4 List the default databases	15
5.5 Create a database user	16
5.6 Create a database, a table, and insert data	16
5.7 Query the database from the VM	16
5.8 Allow remote access from the Linux host	17
5.9 Query the database from the Linux host	17
5.10 List databases with their owners	18

5.11 Inspect the pg_shadow table	18
6. Install PHP	19
6.1 Install the package	19
6.2 Create a test page	19
6.3 View the page from the host	20
7. Install PhpPgAdmin	21
7.1 Install the package	21
7.2 Adjust the Apache configuration	21
7.3 Access PhpPgAdmin from the host	22
8. Deploy a PHP page	23
8.1 Copy the file into your VM	23
8.2 View the page from the host	23
9. Check storage usage	24
9.1 List block devices and filesystems	24
9.2 Check disk usage	24
10. Security analysis	25
10.1 Debian base system	25
10.2 Apache	25
10.3 PHP	25
10.4 PostgreSQL	25
10.5 The big picture	26
Appendix A — Useful commands	27
Appendix B — Troubleshooting	28
Apache won't start	28
Cannot connect to PostgreSQL from the host	28
Port forwarding doesn't work	28
phpinfo() page is blank or downloads instead of running	28
Appendix C — References	29

About this guide

This guide shows you how to install a Debian 13 web server inside a Qemu/KVM virtual machine. By the end, you'll have a working server running Apache, PostgreSQL, and PHP. You'll be able to reach it from your host machine, just like any real web server.

This guide is written for readers who are comfortable with the Linux command line and have basic familiarity with system administration. You should already know how to:

- use a Linux shell and run basic commands as a regular user or as root
- write simple SQL queries (SELECT, INSERT, CREATE TABLE)
- read a configuration file and edit it with a text editor such as nano or vim

You'll need around three to four hours to complete every step.

Note — This guide doesn't cover advanced topics like SSH key authentication, HTTPS with TLS 1.3, or production hardening. It focuses on a clean, working installation, a solid foundation if you want to go further later.

Conventions used in this guide

A few visual cues will help you follow along.

Commands you type appear in a gray box. The prompt tells you which account to use:

```
$ command    # regular user account
# command    # root account
```

File paths and configuration keys look like this: `/etc/apache2/apache2.conf`.

Three kinds of callouts may appear next to a step:

Note — Useful context or background.

Caution — Something to double-check before moving on.

Warning — A step that may break your installation if you skip or rush it.

1. Before you start

1.1 What you need

Here's everything you should have ready before you boot the installer.

Required	Details
Host machine	A Linux station with Qemu/KVM installed
Installer image	debian-13.x-amd64-netinst.iso
Disk space	At least 10 GB free on the host
Network	Internet access for apt package downloads
Time	Around 3 to 4 hours

The Debian ISO is available from `/usr/local/images-ISO/` if your environment provides it as a shared image, otherwise grab it from the official Debian website.

1.2 A note on the helper scripts

Throughout this guide, you'll see commands like:

```
$ S2.03-lance-installation
$ S2.03-lance-machine-virtuelle
$ S2.03-deplace-image-disque-sur-erebus
```

These scripts are wrappers around `qemu-system-x86_64`. They include the options needed for the local setup: port forwarding, KVM acceleration, memory, and so on. If you don't have them, you can run `qemu-system-x86_64` directly with the same flags. The rest of the guide will still work. .

Note — If you want to recreate this VM from scratch (for example, on a USB stick), you will need to write your own `qemu` command. Most `qemu`/KVM tutorials online cover the flags you need. The rest of this guide stays the same.

2. Install Debian 13

Now the interesting part. You are going to install a fresh Debian 13 server inside a virtual machine. There will be no graphical interface, since this is a real server, and servers do not need a desktop that nobody will use.

2.1 Verify the installer image

Before you use the ISO, you should check that the file is not corrupted. A corrupted ISO can fail in unexpected ways during the installation, and you may spend a lot of time trying to understand the problem.

Open the SHA-512 checksums published by Debian:

```
https://cdimage.debian.org/cdimage/release/current/amd64/iso-cd/
```

Look for the file named SHA512SUMS. Find the line matching your ISO filename.

Now compute the checksum of your local file:

```
$ sha512sum /usr/local/images-ISO/debian-13.x-amd64-netinst.iso
```

Compare the two values. They should match character for character. If they don't, the file is corrupted — get a fresh copy before going any further.

Note — SHA-512 is a hash function. It turns any file into a 128-character fingerprint. Change one bit in the file and the fingerprint changes completely. That's why this check works.

2.2 Boot the installer

Launch the helper script:

```
$ S2.03-lance-installation
```

The virtual machine starts and boots from the ISO. You'll see the Debian installer menu. Pick "Install" (text mode) and press Enter.

2.3 Configure language and location

The installer asks you a series of questions. Use these values:

- **Language:** English
- **Location:** other → Europe → France
- **Locales:** United States — en_US.UTF-8
- **Keyboard:** French

Note — Setting the language to English may feel strange when you're in France, but it keeps system messages and log files in English which makes them way easier to search for online.

2.4 Set the hostname and user account

When the installer asks for a hostname, use `server-YOURLOGIN` — replace YOURLOGIN with the login you want to use. This is how the machine identifies itself on the network.



Then set the passwords and user account:

- **Root password:** a simple password works for an initial setup. Check "Show Password" so you can see what you type.
- **Full name:** your real name (e.g. "Jean Toto")
- **Username:** the login you want to use (lowercase, no spaces)
- **User password:** a simple password, also visible via "Show Password"

Warning — In a real production system, you'd never use a simple password or display it on screen. Keep this in mind : the choices below are made to get a working setup quickly, not for facing the public internet.

2.5 Partition the disk

Use the simplest partitioning scheme available:

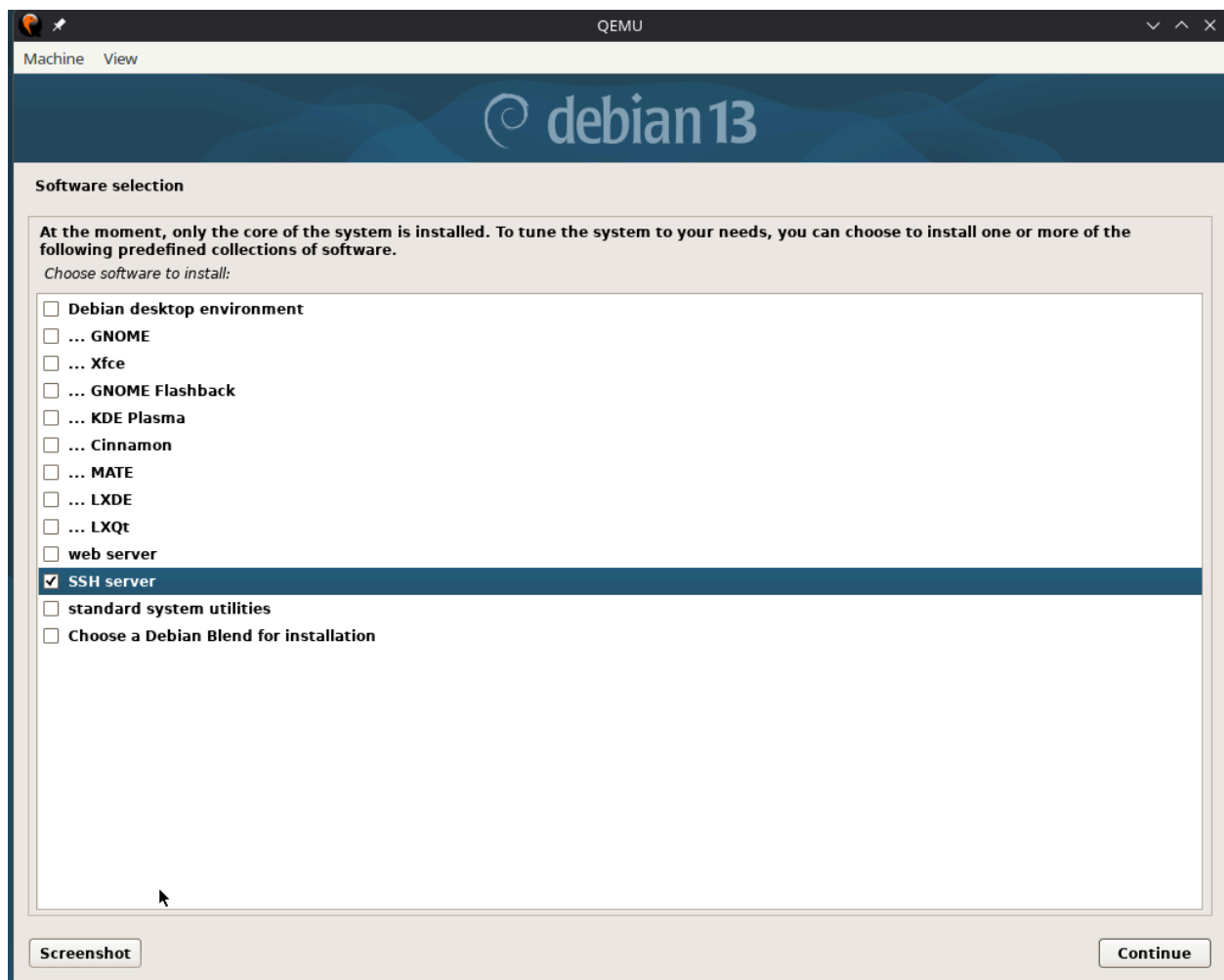
- **Partitioning method:** Guided — use entire disk
- **Partitioning scheme:** All files in one partition
- **Write changes:** Yes

2.6 Select the software

This is the most important step of the install. Pay attention here.

The installer shows a list of "software collections" to install. By default, "Debian desktop environment" is checked. You don't want a desktop, this is a server.

- Uncheck **Debian desktop environment** (and any GNOME / KDE / Xfce sub-items)
- Make sure **SSH server** is checked
- Make sure **standard system utilities** is checked



Caution — If you forget to uncheck the desktop, the installer will pull in over 1 GB of packages you don't need, and your VM will boot into a graphical login screen instead of a console.

2.7 Install GRUB and finish

Two final questions:

- **Install GRUB bootloader:** Yes
- **Device for boot loader:** /dev/sda

When the installer finishes, the VM will reboot. You need to shut it down properly before continuing. The script used to launch the installer is not the same as the one used to start the installed system.

Log in as root and run:

```
# systemctl poweroff
```

Note — Always use `systemctl poweroff` to stop your VM. Killing the `qemu` process directly can corrupt the disk image.

2.8 Move the disk image to the shared server

The disk image was created on your local Linux station. Move it to the shared server so you can launch your VM from any workstation later (and so it survives if your local machine is reset).

```
$ S2.03-deplace-image-disque-sur-erebus
```

You can also copy the image to a USB stick if you want to take it home.

3. Verify the base system

Before you start installing services, let's check that the base system is clean and behaving as expected.

3.1 Start the VM

```
$ S2.03-lance-machine-virtuelle
```

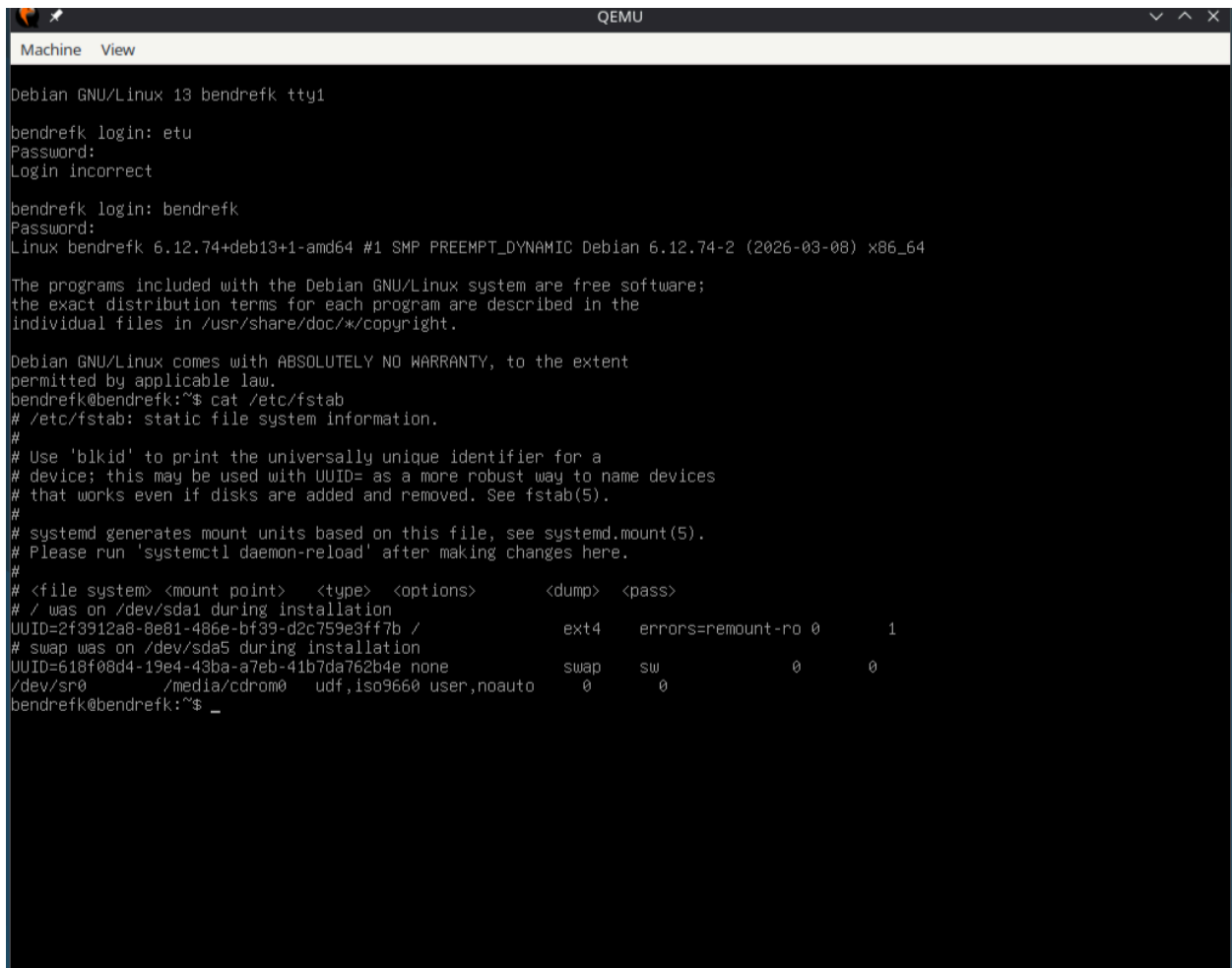
Log in as your regular user with the password you set during the install.

3.2 Check the filesystem table

The `/etc/fstab` file describes how partitions are mounted at boot. Take a look:

```
$ cat /etc/fstab
```

You should see one main partition mounted as `/` (root), plus a swap entry.



```
QEMU
Machine View
Debian GNU/Linux 13 bendrefk tty1
bendrefk login: etu
Password:
Login incorrect

bendrefk login: bendrefk
Password:
Linux bendrefk 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

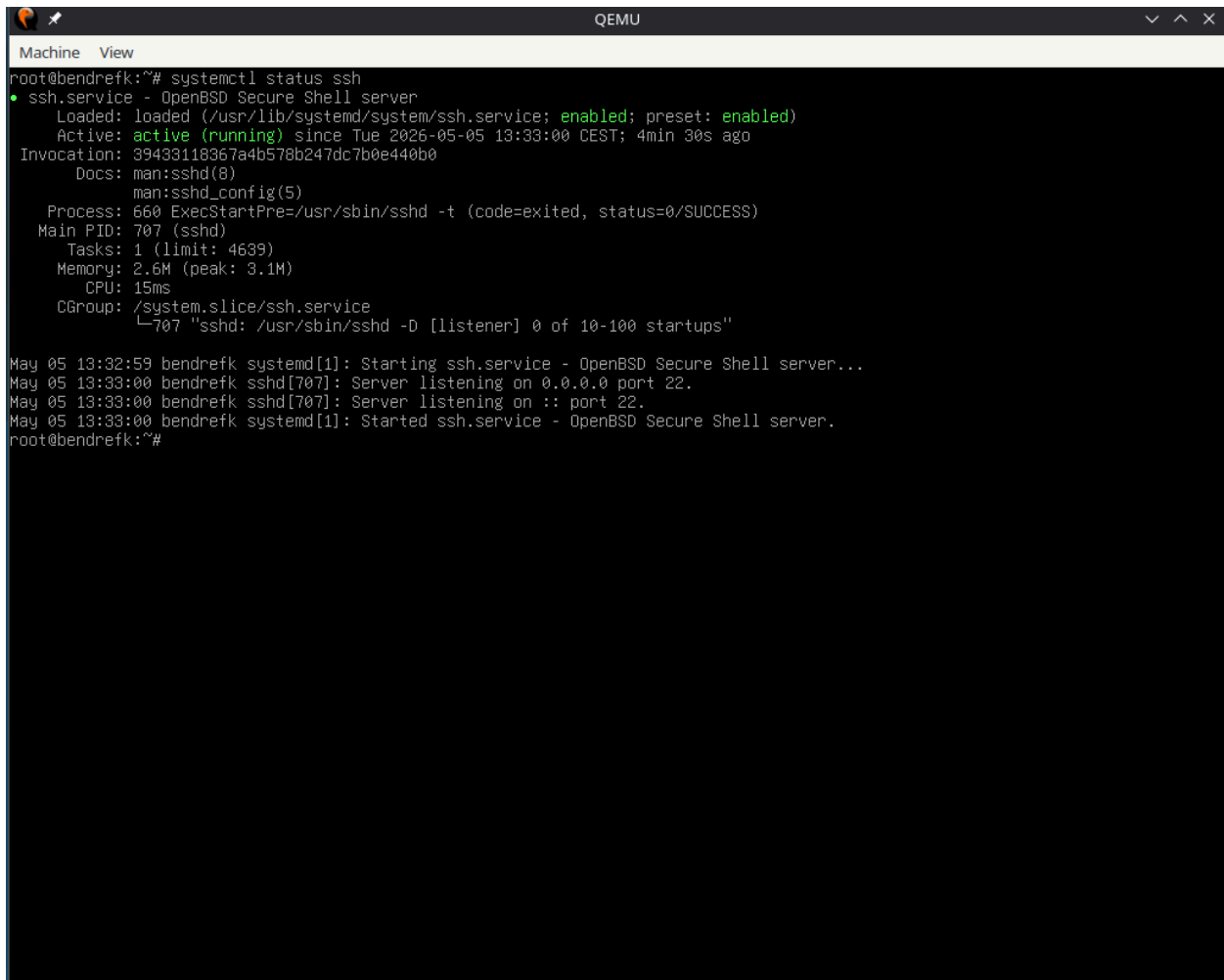
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
bendrefk@bendrefk:~$ cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# systemd generates mount units based on this file, see systemd.mount(5).
# Please run 'systemctl daemon-reload' after making changes here.
#
# <file system> <mount point> <type> <options> <dump> <pass>
# / was on /dev/sda1 during installation
UUID=2f3912a8-8e81-486e-bf39-d2c759e3ff7b / ext4 errors=remount-ro 0 1
# swap was on /dev/sda5 during installation
UUID=618f08d4-19e4-43ba-a7eb-41b7da762b4e none swap sw 0 0
/dev/sr0 /media/cdrom0 udf,iso9660 user,noauto 0 0
bendrefk@bendrefk:~$ _
```

3.3 Check that SSH is running

The SSH server should already be running — you installed it during the Debian setup.

```
# systemctl status ssh
```

The output should show **active (running)** in green. If not, start it with `systemctl start ssh`.



```
Machine View
root@bendrefk:~# systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-05-05 13:33:00 CEST; 4min 30s ago
   Invocation: 39433118367a4b578b247dc7b0e440b0
   Docs: man:sshd(8)
         man:sshd_config(5)
   Process: 660 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 707 (sshd)
   Tasks: 1 (limit: 4639)
   Memory: 2.6M (peak: 3.1M)
   CPU: 15ms
   CGroup: /system.slice/ssh.service
           └─707 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

May 05 13:32:59 bendrefk systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
May 05 13:33:00 bendrefk sshd[707]: Server listening on 0.0.0.0 port 22.
May 05 13:33:00 bendrefk sshd[707]: Server listening on :: port 22.
May 05 13:33:00 bendrefk systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@bendrefk:~#
```

3.4 Confirm there's no graphical interface

A server shouldn't have X11 or Wayland installed. Check both:

```
$ dpkg -l | grep libx11
$ dpkg -l | grep wayland
```

If either command lists packages, you accidentally installed a desktop. You can purge those packages, but the cleaner fix is to reinstall Debian and uncheck the desktop properly in step 2.6.

Note — Why no GUI? A graphical interface eats memory, opens extra network ports, and gives an attacker more code to exploit.

3.5 Port forwarding overview

The helper script sets up port forwarding between your Linux host and the VM. Here's the map:

Service	Port (VM)	Port (host)	Example
SSH	22	2222	<code>ssh user@localhost -p 2222</code>
HTTP	80	8080	<code>http://localhost:8080/</code>
HTTPS	443	4443	<code>https://localhost:4443/</code>
PostgreSQL	5432	5432	<code>psql -h localhost -U user db</code>

Note — Port forwarding means: when you connect to `localhost:8080` on your host, the connection is silently rerouted to port 80 inside the VM. Your VM doesn't have a real IP that the host can reach directly — this trick is how the host and the guest talk to each other over user-mode networking.

4. Install and configure Apache

Apache is the web server. It listens for HTTP requests and serves files (HTML, images, PHP output) back to the browser.

4.1 Install the package

Log in as root (or use sudo if you've set it up) and run:

```
# apt update
# apt install apache2
```

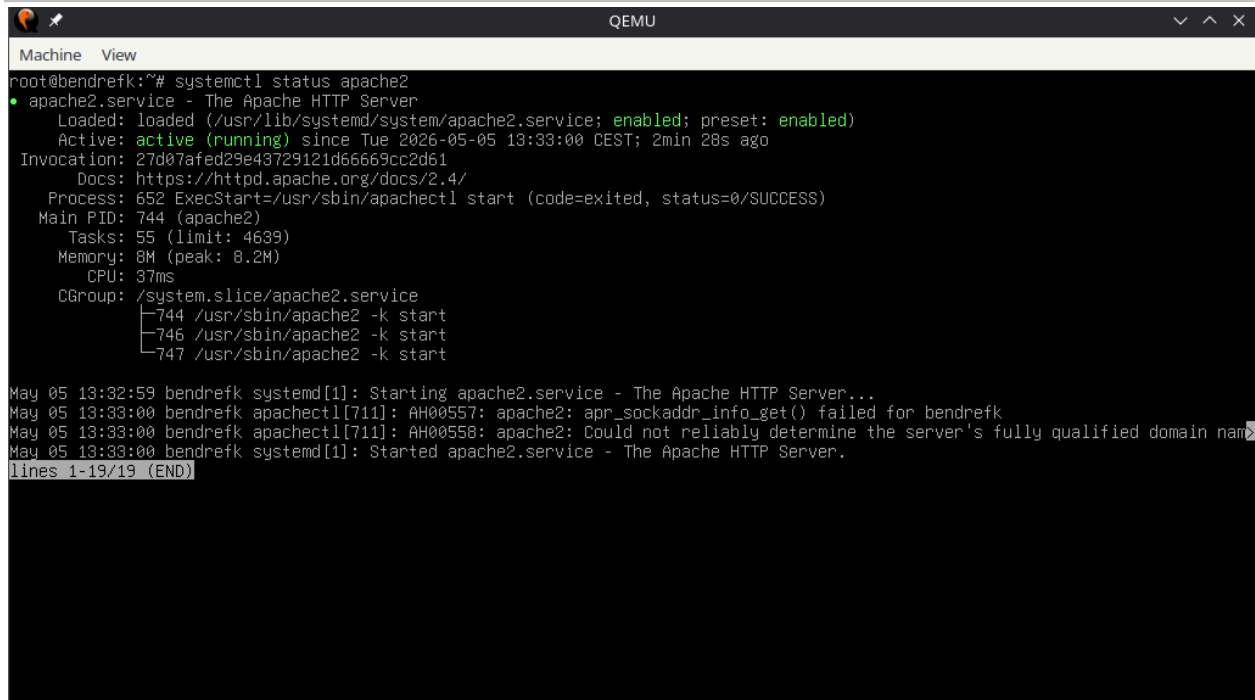
Apt downloads Apache and its dependencies, then starts the service automatically.

4.2 Check the service status

```
# systemctl status apache2
```

Look for **active (running)**. If Apache isn't running for some reason, start it:

```
# systemctl start apache2
```



```
Machine View
root@bendrefk:~# systemctl status apache2
• apache2.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
  Active: active (running) since Tue 2026-05-05 13:33:00 CEST; 2min 28s ago
  Invocation: 27d07afed29e43729121d66669cc2d61
  Docs: https://httpd.apache.org/docs/2.4/
  Process: 652 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 744 (apache2)
  Tasks: 55 (limit: 4639)
  Memory: 8M (peak: 8.2M)
  CPU: 37ms
  CGroup: /system.slice/apache2.service
          └─744 /usr/sbin/apache2 -k start
            └─746 /usr/sbin/apache2 -k start
              └─747 /usr/sbin/apache2 -k start

May 05 13:32:59 bendrefk systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 05 13:33:00 bendrefk apachectl[711]: AH00557: apache2: apr_sockaddr_info_get() failed for bendrefk
May 05 13:33:00 bendrefk apachectl[711]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name
May 05 13:33:00 bendrefk systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-19/19 (END)
```

4.3 Test the server from inside the VM

Since the VM has no browser, you can't visit the page from here. But you can prove the server is listening with netcat:

```
$ netcat -zv localhost 80
```

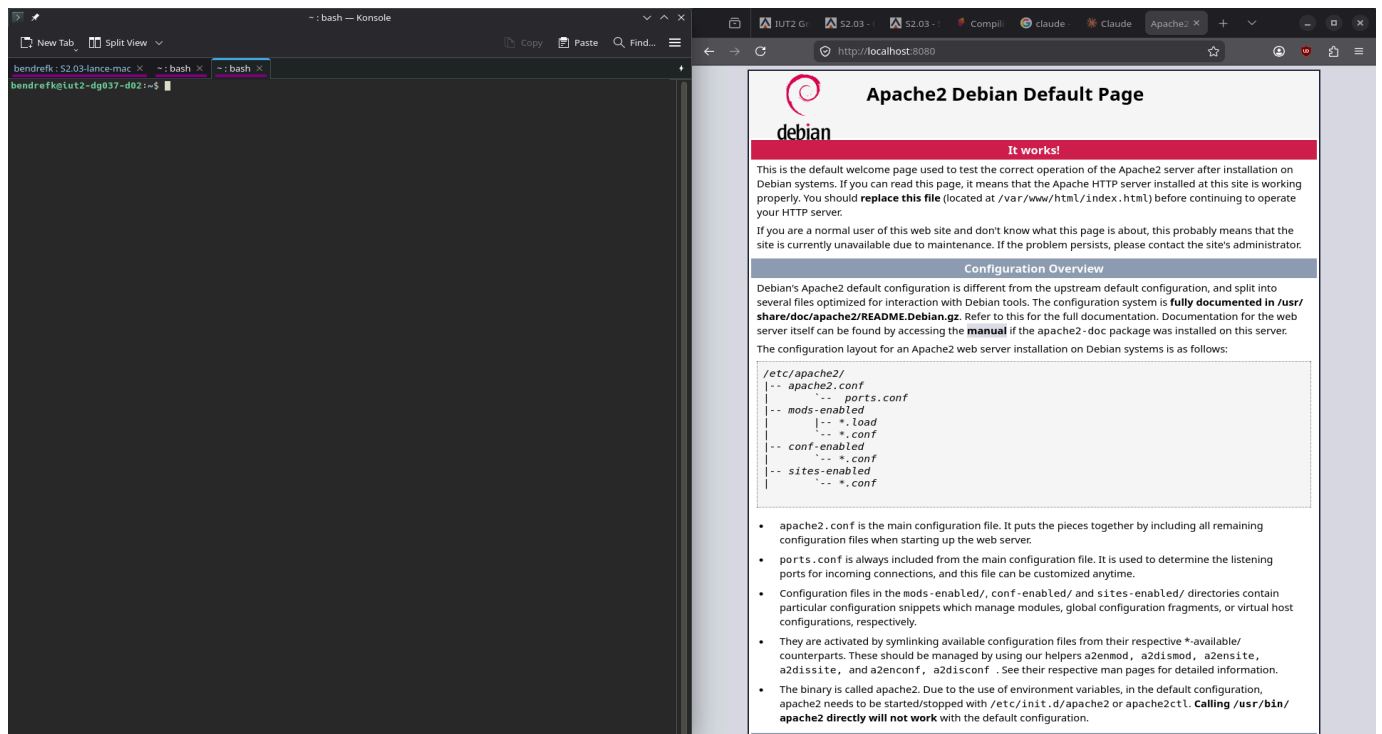
A successful connection prints succeeded or open, depending on the netcat version.

4.4 Test the server from your Linux host

Now open a browser on your Linux station and visit:

```
http://localhost:8080/
```

You should see the default Apache welcome page — "It works!" or the more modern Debian default page.



Note — If the page doesn't load, double-check that your VM is still running and that you used port 8080 (host) and not 80. Port 80 on your host belongs to the host itself, not the VM.

5. Install and configure PostgreSQL

PostgreSQL is the database server. It stores tables, runs SQL queries, and (for our purposes) does anything you'd want a relational database to do.

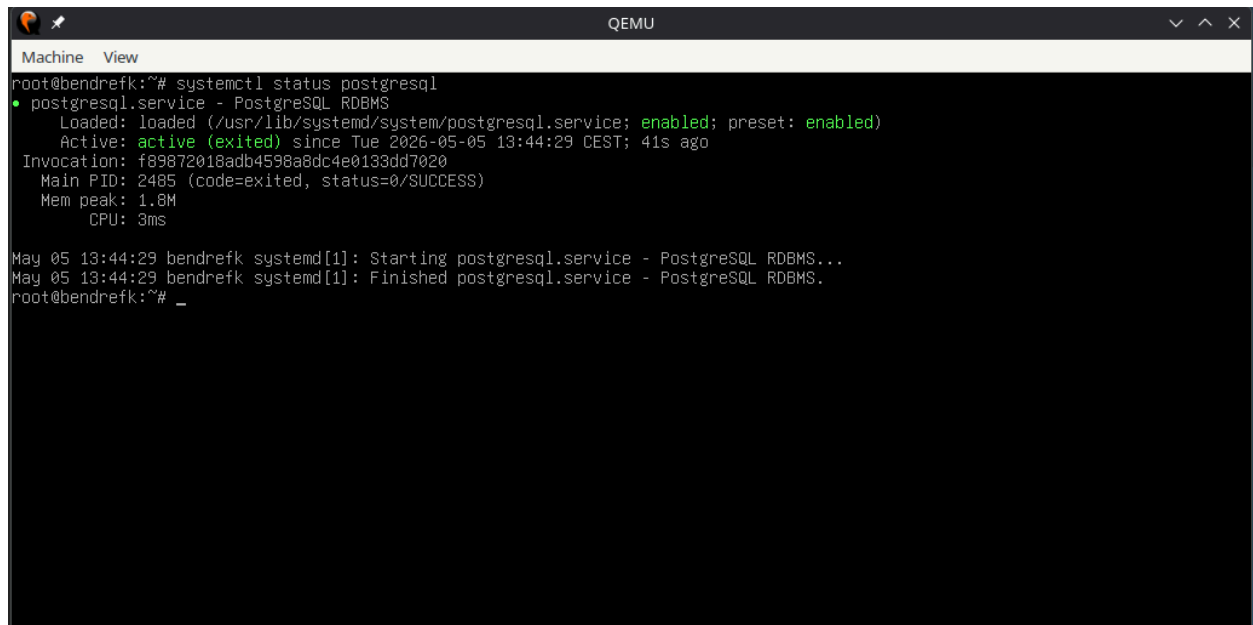
5.1 Install the package

```
# apt install postgresql
```

Apt installs the server, the client (psql), and a few helper tools. The service starts automatically.

5.2 Check the service status

```
# systemctl status postgresql
```

A screenshot of a terminal window titled 'QEMU' showing the output of the command 'systemctl status postgresql'. The output indicates that the postgresql.service is loaded and enabled, and it is currently active (exited) since Tuesday, May 2026-05-05 at 13:44:29 CEST, 41 seconds ago. The terminal also shows logs for the service starting and finishing. The prompt is root@bendrefk:~#.

```
Machine View
root@bendrefk:~# systemctl status postgresql
• postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Tue 2026-05-05 13:44:29 CEST; 41s ago
  Invocation: f89872018adb4598a8dc4e0133dd7020
    Main PID: 2485 (code=exited, status=0/SUCCESS)
   Mem peak: 1.8M
      CPU: 3ms

May 05 13:44:29 bendrefk systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
May 05 13:44:29 bendrefk systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
root@bendrefk:~# _
```

5.3 Switch to the postgres user

PostgreSQL creates a system user called postgres during install. This is the database superuser. Switch to it:

```
# su - postgres
```

Your prompt should change to postgres@server-yourlogin:~\$.

5.4 List the default databases

```
$ psql -l
```

You'll see three default databases: postgres, template0, and template1.

5.5 Create a database user

From the postgres shell, connect to psql and create a user:

```
$ psql
postgres=# CREATE USER yourlogin WITH PASSWORD 'password';
```

Replace yourlogin with the login of your choice.

5.6 Create a database, a table, and insert data

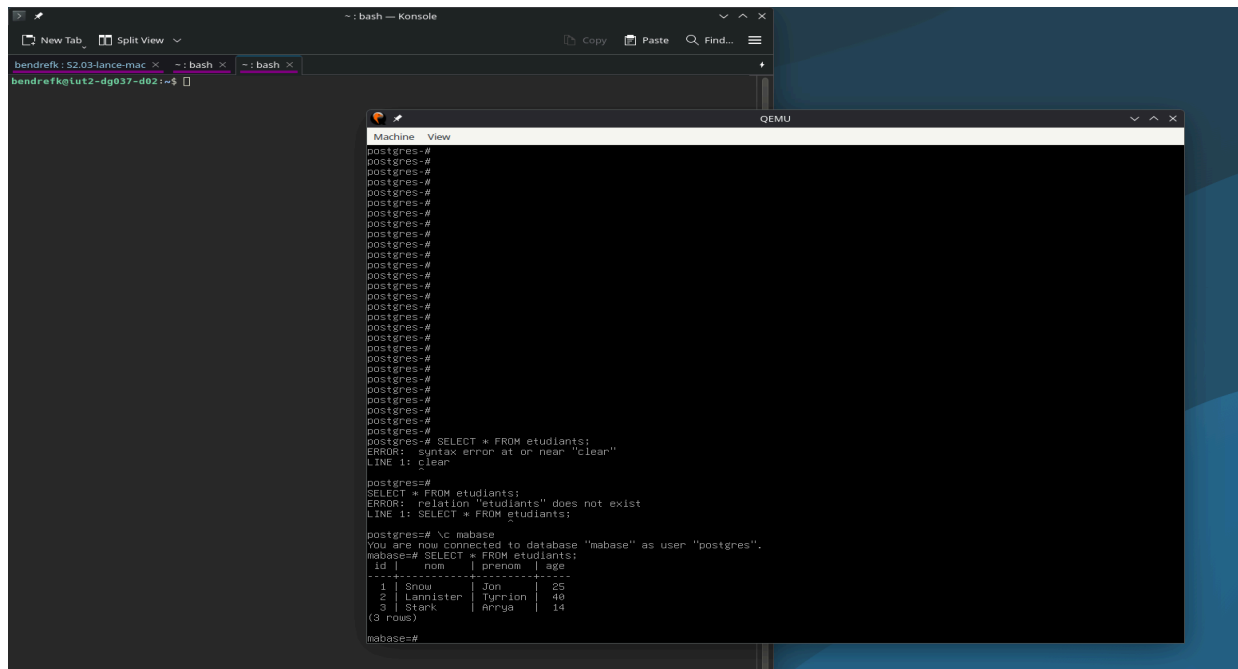
Still inside psql:

```
postgres=# CREATE DATABASE mybase OWNER yourlogin;
postgres=# \c mybase yourlogin
mybase=> CREATE TABLE students (
mybase->   id SERIAL PRIMARY KEY,
mybase->   surname VARCHAR, name VARCHAR, year INT);
mybase=> INSERT INTO students (surname, name, year) VALUES
mybase->   ('Snow', 'Jon', 25),
mybase->   ('Lannister', 'Tyrrion', 40),
mybase->   ('Stark', 'Arrya', 14);
```

5.7 Query the database from the VM

Now run a query to make sure everything works:

```
mybase=> SELECT * FROM students;
```



Exit psql with \q and return to the regular shell with exit.

5.8 Allow remote access from the Linux host

For now, PostgreSQL only accepts local connections in peer mode. This means that the postgres Unix user can connect from inside the VM, but no connection from outside is accepted. To allow remote (TCP) connections from users authenticated by a password, you need to edit two configuration files.

First, edit `postgresql.conf`:

```
# nano /etc/postgresql/17/main/postgresql.conf
```

Find the line `#listen_addresses = 'localhost'` and change it to:

```
listen_addresses = '*'
```

Note — The version number in the path (17 here) depends on the PostgreSQL release that comes with Debian 13. If apt installed a different version, change the path accordingly. Inside nano, use Ctrl-W to find the `listen_addresses` line, then uncomment it and set the value to `'*'`. Use Ctrl-O to save and Ctrl-X to quit.

Next, edit `pg_hba.conf` — this file controls who can connect and how:

```
# nano /etc/postgresql/17/main/pg_hba.conf
```

Add this line under the "IPv4 remote connections" section. It allows connections from any IPv4 address, as long as the user is authenticated by a password using the secure SCRAM-SHA-256 mechanism. The postgres superuser still has no password defined, so this rule does not give it remote access:

```
host      all      all      0.0.0.0/0      scram-sha-256
```

Save and exit. Switch back to root (logout from the postgres shell) and restart the service so both files are reloaded:

```
# systemctl restart postgresql
```

5.9 Query the database from the Linux host

From your Linux station (not the VM), run:

```
$ psql -h localhost -p 5432 -U yourlogin -d mybase
```

You'll be prompted for the password you set in step 5.5. Once connected, run the same query:

```
mybase=> SELECT * FROM students;
```

```
bendrefk@lut2-dg037-d02:~$ psql -h localhost postgres
Password for user bendrefk:
psql (17.9 (Debian 17.9-0+deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> \c mabase
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "mabase" as user "bendrefk".
mabase=> SELECT * FROM etudiants;
 id |  nom  | prenom | age
-----+-----+-----+---
  1 | Snow  | Jon    | 25
  2 | Lannister | Tyrrion | 40
  3 | Stark | Arrya  | 14
(3 rows)

mabase=> █
```

5.10 List databases with their owners

Back inside psql (either from the VM or from the host), list all databases together with the user who owns them:

```
mybase=# \l
```

```
bendrefk@lut2-dg037-d02:~$ psql -h localhost postgres
Password for user bendrefk:
psql (17.9 (Debian 17.9-0+deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> \c mabase
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "mabase" as user "bendrefk".
mabase=> SELECT * FROM etudiants;
 id |  nom  | prenom | age
-----+-----+-----+---
  1 | Snow  | Jon    | 25
  2 | Lannister | Tyrrion | 40
  3 | Stark | Arrya  | 14
(3 rows)

mabase=> \l

      Name | Owner  | Encoding | Locale Provider | Collate | Ctype | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 mabase   | bendrefk | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |         |          | 
 postgres | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |         |          | 
 template0 | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |         |          | =c/postgres +
                                         postgres=Ctc/postgres
 template1 | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |         |          | =c/postgres +
                                         postgres=Ctc/postgres
(4 rows)

mabase=> █
```

5.11 Inspect the pg_shadow table

PostgreSQL stores user passwords in a system table called `pg_shadow`. The passwords are stored with SCRAM-SHA-256, a secure hashing scheme based on SHA-256, never stored in plain text.

You need to be a superuser to read this table. Switch to the postgres user, then run:

```
mybase=# SELECT * FROM pg_shadow;
```

```
bendrefk@lnt2-dg937-d92:~$ psql -h localhost postgres
psql (17.9 (Debian 17.9-4deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type 'help' for help.

postgres=> \c mabase
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "mabase" as user "bendrefk".
mabase=> SELECT * FROM etudiants;
ERROR: permission denied for table etudiants
mabase=> \c
postgres=> \l
bendrefk@lnt2-dg937-d92:~$ psql -h postgres@localhost postgres
psql: error: could not translate host name "postgres@localhost" to address: Name or service not known
bendrefk@lnt2-dg937-d92:~$ psql -h localhost postgres
Password for user bendrefk:
psql: error: connection to server at "localhost" (::1), port 5432 failed: Connection refused
Is the server running on that host and accepting TCP/IP connections?
connection to server at "localhost" (127.0.0.1), port 5432 failed: could not initiate GSSAPI security context: Unspecified GSS failure. Minor code may provide more information: Server not found in Kerberos database
connection to server at "localhost" (127.0.0.1), port 5432 failed: FATAL: password authentication failed for user "bendrefk"
connection to server at "localhost" (127.0.0.1), port 5432 failed: FATAL: password authentication failed for user "bendrefk"
bendrefk@lnt2-dg937-d92:~$ psql -h localhost postgres
Password for user bendrefk:
psql (17.9 (Debian 17.9-4deb13u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type 'help' for help.

postgres=> \c mabase
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "mabase" as user "bendrefk".
mabase=> SELECT * FROM etudiants;
 id |  nom  | prenom | age
----+-----+-----+----
 1 | Snow  | Jon    | 25
 2 | Lannister | Tyrion | 40
 3 | Stark | Arya   | 14
(3 rows)

mabase=> \l
      Name | Owner | Encoding | Locale Provider | Collate | Ctype | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 mabase   | bendrefk | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 | | | |
 postgres | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 | | | |
 template0 | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 | | | |
 template1 | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 | | | |
(4 rows)

mabase=> SELECT * FROM pg_shadow;
ERROR: permission denied for view pg_shadow
mabase=> SELECT * FROM pg_shadow;
 username | usesysid | usecreatedb | usesuper | usebypassrls |          passwd          |          |          | useconfig
-----+-----+-----+-----+-----+-----+-----+-----+-----
 postgres | 10      | t           | t         | t             |                          |          |          |
 bendrefk | 16388   | f           | f         | f             |                          |          |          |
(2 rows)

mabase=>
```

6. Install PHP

PHP turns Apache from a static file server into a dynamic application server. When a browser requests a .php file, Apache hands it to PHP, PHP runs the code, and the output is sent back to the browser as HTML.

6.1 Install the package

```
# apt install php libapache2-mod-php
```

The libapache2-mod-php package is the glue that lets Apache run PHP. Restart Apache so it picks up the new module:

```
# systemctl restart apache2
```

6.2 Create a test page

Create a file at /var/www/html/info.php with the following content:

```
<?php
phpinfo();
phpinfo(INFO_MODULES);
?>
```

The phpinfo() function prints a long page describing your PHP installation — useful for debugging, dangerous to leave on a production server.

6.3 View the page from the host

Open your browser at:

```
http://localhost:8080/info.php
```

You should see a big purple table with information about PHP, modules, environment variables, and more.

Warning — Delete info.php once you're done testing. Leaving it on a real server is a textbook security mistake, it tells attackers exactly which PHP version and modules you're running.

7. Install PhpPgAdmin

PhpPgAdmin is a web interface for PostgreSQL. It's the PostgreSQL equivalent of phpMyAdmin, and it lets you browse and edit your databases from a browser instead of psql.

7.1 Install the package

```
# apt install phppgadmin
```

Apt installs PhpPgAdmin and configures Apache to serve it at /phppgadmin.

7.2 Adjust the Apache configuration

By default, the Apache configuration for PhpPgAdmin restricts access to the local machine. Since you want to use PhpPgAdmin from your host, open the configuration file and update the access rules:

```
# nano /etc/apache2/conf-available/phppgadmin.conf
```

Look for a `Require` directive and adjust it. The simplest option is:

```
Require all granted
```

Caution — "Require all granted" is the simplest option, but it's also the most permissive: anyone who can reach your server can access PhpPgAdmin and try to brute-force passwords. Restrict it by IP or hide it behind HTTP basic auth before exposing the server to a wider network.

Restart Apache:

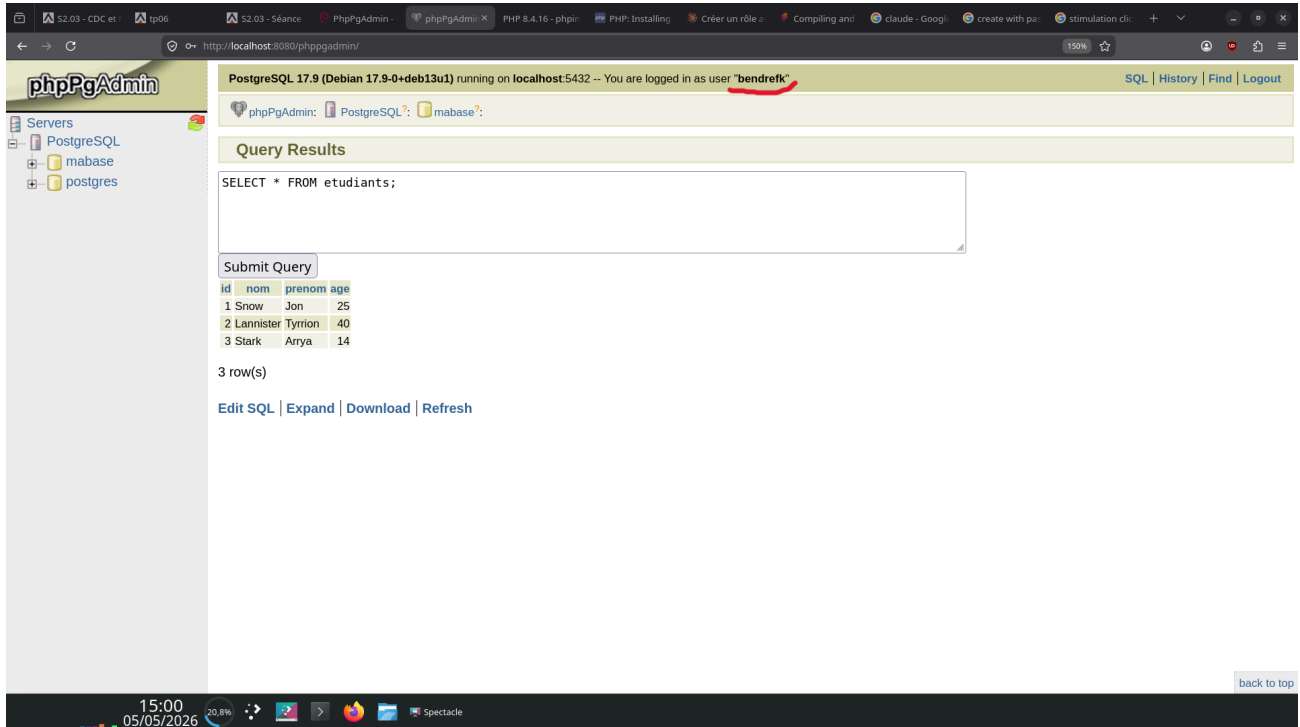
```
# systemctl restart apache2
```

7.3 Access PhpPgAdmin from the host

Open your browser at:

`http://localhost:8080/phpPgAdmin/`

Log in with the PostgreSQL user you created in step 5.5. Navigate to your database, open the students table, and run a query.



The screenshot shows the phpPgAdmin web interface in a browser. The top status bar indicates "PostgreSQL 17.9 (Debian 17.9.0+deb13u1) running on localhost:5432 -- You are logged in as user 'bendrefk'". The left sidebar shows a tree view with "Servers" > "PostgreSQL" > "mabase" > "postgres". The main area displays "Query Results" for the query "SELECT * FROM etudiants;". The results are shown in a table with 3 rows and 4 columns: id, nom, prenom, and age.

id	nom	prenom	age
1	Snow	Jon	25
2	Lannister	Tyrion	40
3	Stark	Arrya	14

3 row(s)

Buttons: Edit SQL | Expand | Download | Refresh

System tray at the bottom shows the time 15:00 on 05/05/2026 and battery level 20.8%.

8. Deploy a PHP page

Now that PHP works, let's deploy a real PHP page on your server. Any PHP file works for this; in this guide we use a page available at:

/users/info/www/intranet/enseignements/S2.03/page_sae_S2.03.php

8.1 Copy the file into your VM

From your Linux host, copy the file into the VM using `scp` over the SSH port forward:

```
$ scp -P 2222 /users/info/www/intranet/enseignements/S2.03/page_sae_S2.03.php  
yourlogin@localhost:/tmp/
```

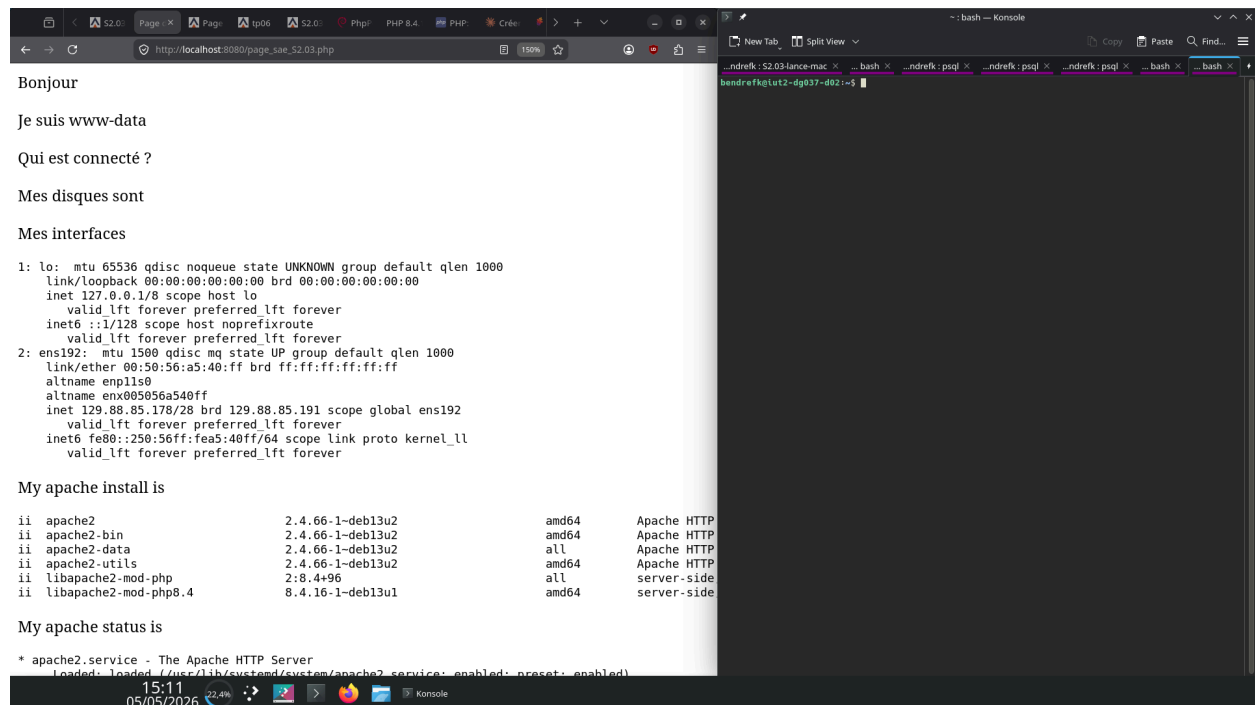
Then SSH into the VM and move it to the Apache document root:

```
$ ssh yourlogin@localhost -p 2222
$ su -
# mv /tmp/page_sae_S2.03.php /var/www/html/
# chown www-data:www-data /var/www/html/page_sae_S2.03.php
```

8.2 View the page from the host

Open your browser at:

http://localhost:8080/page_sae_S2.03.php



9. Check storage usage

Two quick checks to confirm your VM has enough room left to keep working.

9.1 List block devices and filesystems

The `blkid` command lists block devices on the VM, with their UUIDs and filesystem types. Run it as root:

```
# blkid
```

You should see one or two entries — the main partition (ext4) and possibly a swap partition.

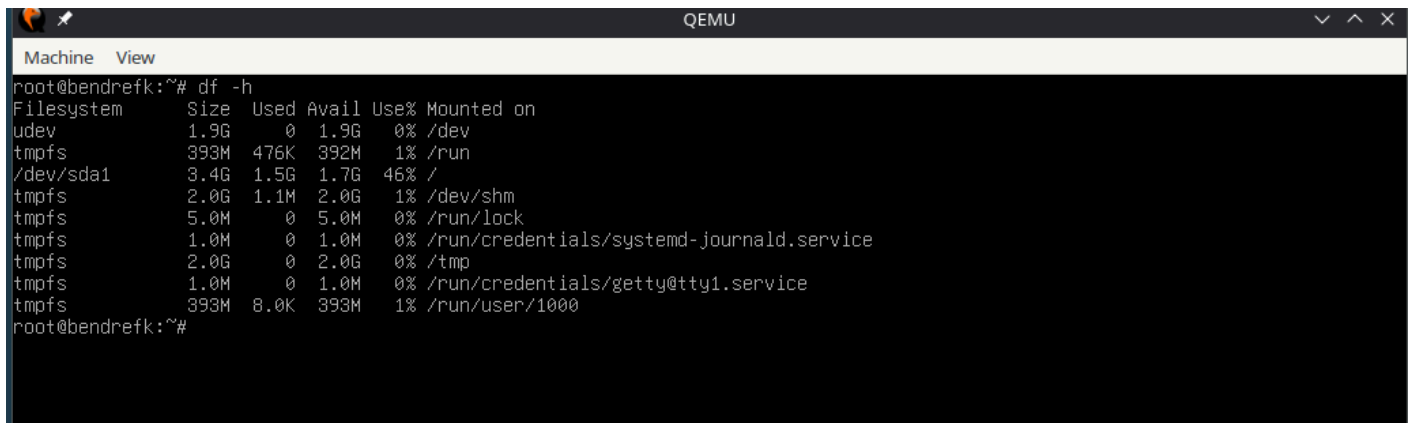
9.2 Check disk usage

The `df -h` command shows used and available space on every mounted filesystem, in human-readable units:

```
$ df -h
```

Look at the line for `/` (the root filesystem). The "Used" and "Avail" columns tell you how much room your installation is taking and how much is left.

Note — It's convenient to run both `blkid` and `df -h` in the same terminal session. A single screenshot then captures the block-device layout and the available space at once.



The screenshot shows a terminal window titled "QEMU" with a "Machine View" tab. The terminal output is as follows:

```
root@bendrefk:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            1.9G     0  1.9G   0% /dev
tmpfs           393M  476K  392M   1% /run
/dev/sda1       3.4G  1.5G  1.7G  46% /
tmpfs           2.0G  1.1M  2.0G   1% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           2.0G     0  2.0G   0% /tmp
tmpfs           1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs          393M   8.0K  393M   1% /run/user/1000
root@bendrefk:~#
```

10. Security analysis

This installation works well as a development setup, but it is not ready for the public internet. Below is a list of the main weaknesses and how you would fix them in a real deployment.

10.1 Debian base system

- **Weakness** : Simple, visible passwords for both root and the regular user.
- **Solution** : Use long, random passwords stored in a password manager. A better option is to disable password login over SSH and use key-based authentication.
- **Weakness** : No automatic security updates.
- **Solution** : Install unattended-upgrades and configure it to apply security patches nightly.

10.2 Apache

- **Weakness** : Apache reveals its version and OS in HTTP response headers ("Apache/2.4.x (Debian)").
- **Solution** : Set ServerTokens Prod and ServerSignature Off in /etc/apache2/conf-enabled/security.conf. Attackers can no longer match your server against known CVEs at a glance.
- **Weakness** : HTTP only, traffic isn't encrypted, so passwords and session cookies travel in clear text.
- **Solution** : Enable HTTPS with a valid TLS certificate. For internal services, a self-signed cert is fine; for public services, use Let's Encrypt.

10.3 PHP

- **Weakness** : info.php exposes the full PHP configuration.
- **Solution** : Delete info.php after testing. Set expose_php = Off in /etc/php/*/apache2/php.ini to remove the X-Powered-By header.
- **Weakness** : Default error display is verbose, which can leak file paths and database credentials in error messages.
- **Solution** : Set display_errors = Off in php.ini for production. Log errors to a file instead.

10.4 PostgreSQL

- **Weakness** : pg_hba.conf currently allows password-authenticated connections from any IPv4 address (0.0.0.0/0). It works, but it's as permissive as it gets: anyone who can reach port 5432 may try to log in.

- **Solution** : Narrow the CIDR range to the addresses that actually need access (for example 10.0.2.2/32 for the qemu host, or your internal LAN range). Use strong passwords. Consider client certificates for authentication.
- **Weakness** : PhpPgAdmin is publicly accessible from the host with "Require all granted".
- **Solution** : Restrict PhpPgAdmin by IP, put it behind HTTP basic auth, or just don't install it on a production server — use psql or pgAdmin over SSH tunnels instead.

10.5 The big picture

These weaknesses, taken one by one, will not allow an attacker to enter the server. But security works in layers. A server with a weak password, an exposed admin panel, an unencrypted connection, and verbose error pages is much easier to attack than a server where every layer is protected.

The good news is that almost every fix above is one or two lines in a configuration file. Hardening a Debian server is not really difficult.

Appendix A — Useful commands

A quick reference for the commands used in this guide.

Command	What it does
<code>systemctl status <service></code>	Show whether a service is running
<code>systemctl start <service></code>	Start a service
<code>systemctl restart <service></code>	Reload a service after a config change
<code>systemctl poweroff</code>	Shut the VM down cleanly
<code>apt install <package></code>	Install a package as root
<code>dpkg -l grep <name></code>	Check if a package is installed
<code>cat /etc/fstab</code>	Show the filesystem table
<code>blkid</code>	List block devices and UUIDs (root)
<code>df -h</code>	Show disk usage in human-readable form
<code>netcat -zv localhost <port></code>	Test if a TCP port is open
<code>su - postgres</code>	Switch to the postgres system user
<code>psql -l</code>	List databases
<code>psql -h <host> -U <user> -d <db></code>	Connect to a remote database

Appendix B — Troubleshooting

Apache won't start

Run `journalctl -xeu apache2` to see the last error messages. Most often, another process is already using port 80, or there's a syntax error in a config file.

Cannot connect to PostgreSQL from the host

- Double-check `listen_addresses` in `postgresql.conf` is set to `'*'`.
- Double-check the `pg_hba.conf` line you added.
- Restart PostgreSQL after every config change, otherwise the file is read but not applied.

Port forwarding doesn't work

- Make sure the VM is running. `ps aux | grep qemu` on the host should show a running qemu process.
- Verify you're using the host port, not the VM port (8080 not 80, 2222 not 22, etc.).

phpinfo() page is blank or downloads instead of running

Apache doesn't know it should pass `.php` files to PHP. Make sure `libapache2-mod-php` is installed, and run `a2enmod php8.x` then restart Apache.

Appendix C — References

- Debian installation guide — <https://www.debian.org/releases/stable/installmanual>
- Apache HTTP Server documentation — <https://httpd.apache.org/docs/>
- PostgreSQL documentation — <https://www.postgresql.org/docs/>
- PHP manual — <https://www.php.net/manual/en/>